

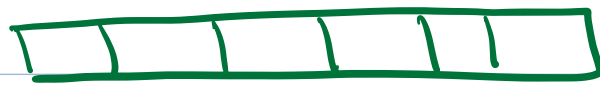
AGENDA

1. 2D Matrix
2. Print row wise/ col wise sum
3. Print Diagonals in a Square Matrix
4. Print Diagonals in a Rec Matrix
5. Row to Column Zero

Contest → 17 Nov

Next Mon

1D arrays : list of elements



2D matrix : 2D array which has a rectangular grid of nos. $\rightarrow$ element

Store elements arranged in row and cols

1. `int mat[N][M]` (C++)

$\downarrow$ data type $\downarrow$ name of 2D mat $\downarrow$ rows $\downarrow$ cols

2. `int[][] mat = new int[N][M]` (Java)

3. `mat = [[0]*M for _ in range(N)]` (Python)

	cols			
	0	1	2	
rows	0	0,0	0,1	0,2
	1	1,0	1,1	1,2
	2	2,0	2,1	2,2
	3	3,0	3,1	3,2

(r, c)

cell
 $\downarrow$
`mat[r][c]`

`int mat[4][3]`
 $\downarrow$ rows $\downarrow$ cols

Total ele
= $4 \times 3 = 12$

int mat [N][M]
rows cols

cols

Top Left

rows

	0	1	2	3	...	M-1	
0	0,0			0,3		0,M-1	Top Right
1				1,3			
2	2,0	2,1	2,2	2,3		2,M-2	
...				3,3			
...				...			
N-1	N-1,0			N-1,3		N-1,M-1	Bottom Right

Bottom Left

Total ele
= $N \times M$

mat [N][M]
rows col

★ Iterate in a row

1. Row no. is fixed
2. Col no. $\rightarrow [0 \quad M-1]$ $M \rightarrow \text{cols}$

★ Iterate in a col

1. Col no. is fixed
2. Row no. $\rightarrow [0 \quad N-1]$ $N \rightarrow \text{rows}$

Given 2D Matrix $[N][M]$, print row wise sum.

mat[3][4]

	0	1	2	3	O/P
0	1	2	3	4	10
1	5	6	7	8	26
2	9	10	11	12	42

Traverse each row and while traversing take sum of elements present in that row

void solve (int A[][], int N, int M) rows cols

for (r = 0 ; r < N ; r++) <

// get sum of rth row

int sum = 0

for (c = 0 ; c < M ; c++) <

sum = sum + A[r][c]

print (sum)

TC: $O(N \times M)$

SC: $O(1)$

Given 2D Matrix $[N][M]$, print col wise sum.

mat[3][4]

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

O/P 15 18 21 24

Traverse each col, while traversing take sum of elements present in that col.

void solve (int A[][], int ^{rows} N, int ^{cols} M)

for (c = 0 ; c < M ; c++) {

// get sum of c^{th} column

int sum = 0

for (r = 0 ; r < N ; r++) {

sum = sum + A[r][c]

print (sum)

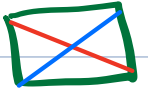
TC : $O(N \times M)$
SC : $O(1)$

rows = cols

3. Given a 2D square matrix $mat[N][N]$, print diagonals

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

$mat[3][3]$



2 diagonals in square matrix

O/P: 1 5 9

$0,0 \rightarrow 1,1 \rightarrow 2,2$

① Principal Diagonal
Top Left $\rightarrow$ Bottom Right

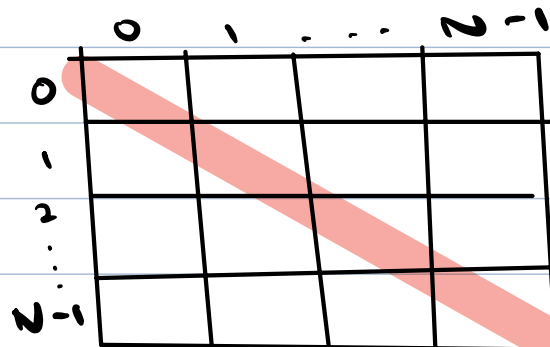
O/P: 3 5 7

② Anti Diagonal

Top Right $\rightarrow$ Bottom Left

$mat[N][N]$

$0,0$
 $\downarrow$
 $1,1$
 $\downarrow$
 $2,2$
 $\downarrow$
 $3,3$
 $\downarrow$
 $\vdots$
 $N-1, N-1$



void printDiagonal (int mat[N][N]) {

```

    int i = 0
    while (i < N) {
        print (mat[i][i])
        i++
    }
}

```

OR

```

for (i = 0 ; i < N ; i++) {
    print (mat[i][i])
}

```

$i \rightarrow [0 \ N-1]$

TC: $O(N)$
SC: $O(1)$

Anti-Diagonal

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

$3 \rightarrow 5 \rightarrow 7$
 $0,2 \rightarrow 1,1 \rightarrow 2,0$

TR

r, c
 $0, N-1$
 $\downarrow$
 $1, N-2$
 $\downarrow$
 $2, N-3$
 $\vdots$

BL

$N-1, 0$
 $\uparrow$
 $\uparrow$

mat[N][N]

	0	...	N-1
0			
1			
2			
...			
N-1			

STOP

$N, -1$

void print Anti Diagonal (int mat[N][N]) <

int r = 0, c = N-1

while (r < N && c >= 0)

print (mat[r][c])

r++ c--

TC: $O(N)$
SC: $O(1)$

No need for 2 conditions, either of 1 condition is sufficient

$(R \rightarrow L)$

Print all anti diagonals of a non-square matrix.

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

O/P

1

2 5

3 6 9

4 7 10

8 11

12

rectangle
rows $\neq$ cols

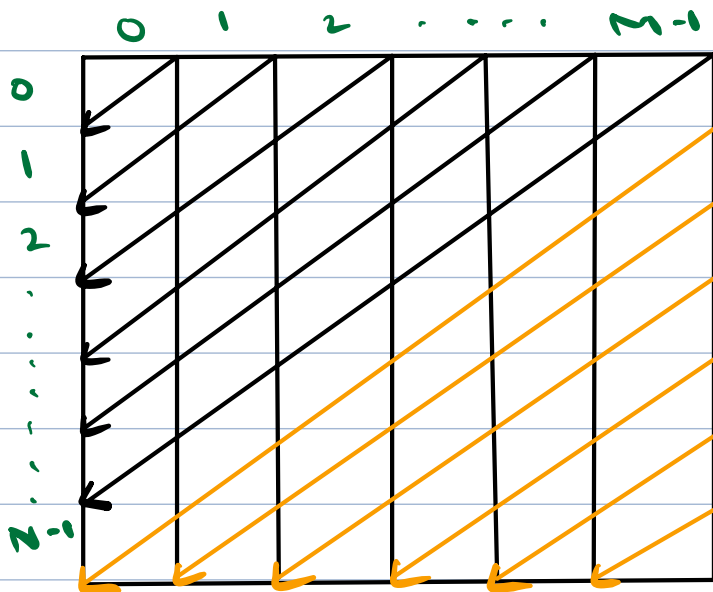
mat [N] [M]

mat[3][4]

cnt = 6

① Every cell in 0th row is starting pt.

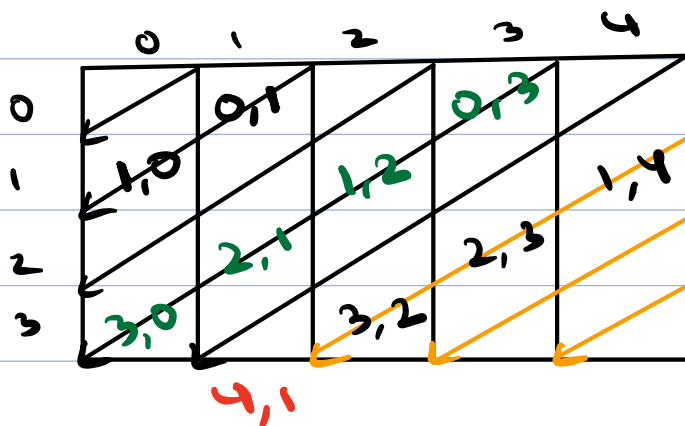
② Every cell in last col is starting pt.



mat[N][M]

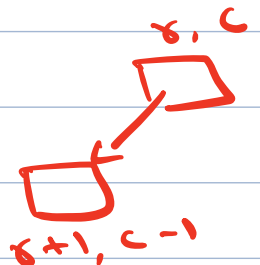
$$\text{cnt} = M + N - 1$$

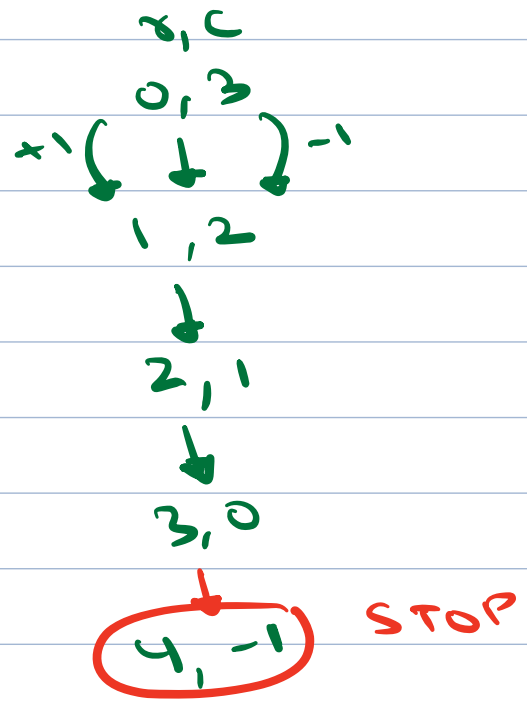
Q. Given mat ^N[4] ^M[5], cnt of right to left (anti) diagonals?



$$\begin{aligned} \text{cnt} &= M + N - 1 \\ &= 5 + 4 - 1 \\ &= 8 \end{aligned}$$

STOP if cell invalid
if r or c is invalid





```

void printAntiDiagonal (int i, int j,
    int mat[N][M]) <

```

```

    int x = i, c = j
    while (x < N && c >= 0) <
        print (mat[x][c])
        x++ c--
    print ("\n")

```

```

// print all diagonals starting at
0th row

```

```

    int row = 0
    for (col = 0; col < M; col++) <
        printAntiDiagonal (row, col, mat)
    }

```

// print all diagonals starting at
 $m-1^{th}$ (last) col

int col = $m-1$

for (row = ~~0~~; row < N ; row++) {
 printAntiDiagonal(row, col, mat)
}

TC: $O(N \times M)$
SC: $O(1)$

Q-5: you have given a 2D array and elements, in the given array if there is any 0 then make entire row and column 0 with respect to that 0 element

	0	1	2	3
0	1	2	3	4
1	5	6	7	0
2	9	2	0	4



O/P:

1	2	0	0
0	0	0	0
0	0	0	0

if $A[i][j] = 0$
make entire
 i^{th} row $\rightarrow 0$
 j^{th} col $\rightarrow 0$

O/P

1	0	6	7
8	9	10	12
13	14	15	2



0	0	0	0
8	0	10	12
13	0	15	2

1 0	0	6 0	7 0
8	9 0	10	12
13	14 0	15	2

0 → originally in mat

0 → your changes

1 -1	0	6 -1	7 -1
8	9 -1	10	12
13	14 -1	15	2



0	0	0	0
8	0	10	12
13	0	15	2

1 -1	0	6 -1	0
8	9 -1	10	12 -1
13	14 -1	15	2 -1



0	0	0	0
8	0	10	0
13	0	15	0

- ① Iterate in all rows, make elements of row as -1, if any element in that row is 0.
- 2 Repeat process for cols
- ③ Finally traverse matrix, make all -1 into 0.

```

void rowToColZero (int mat [N][M]) {
    for (r = 0; r < N; r++) {
        // in rth row, is there any 0?
        bool flag = F
        for (c = 0; c < M; c++) {
            if (mat[r][c] == 0)
                flag = T
        }
        if (flag == T) {
            for (c = 0; c < M; c++) {
                if (mat[r][c] != 0)
                    mat[r][c] = -1
            }
        }
    }

    for (c = 0; c < M; c++) {

```

// in c^{th} col, is there a 0?

bool flag = F

for (x = 0; x < N; x++) <

if (mat[x][c] == 0)

flag = T

if (flag == T) <

for (x = 0; x < N; x++) <

if (mat[x][c] != 0)

mat[x][c] = -1

for (x = 0; x < N; x++) <

for (c = 0; c < M; c++) <

if (mat[x][c] == -1)

mat[x][c] = 0

✓

1 row $\rightarrow 2M$
N row $\rightarrow 2NM$

1 col $\rightarrow 2N$
M col $\rightarrow 2NM$

$2NM +$
 $\Rightarrow 5NM$

$2NM + NM$

TC: $O(NM)$
SC: $O(1)$

int a[5]

Adding an item at end $\rightarrow O(1)$

On avg, adding an element in a dynamic array $\rightarrow O(1)$

ArrayList<int> a

1st element $\rightarrow 1$ ops

2nd $\rightarrow 1$ ops

3rd $\rightarrow 1$ ops

4th $\rightarrow 1$ ops

5th $\rightarrow 1$ ops

6th $\rightarrow 6$ ops

7 $\rightarrow 1$

8 $\rightarrow 1$

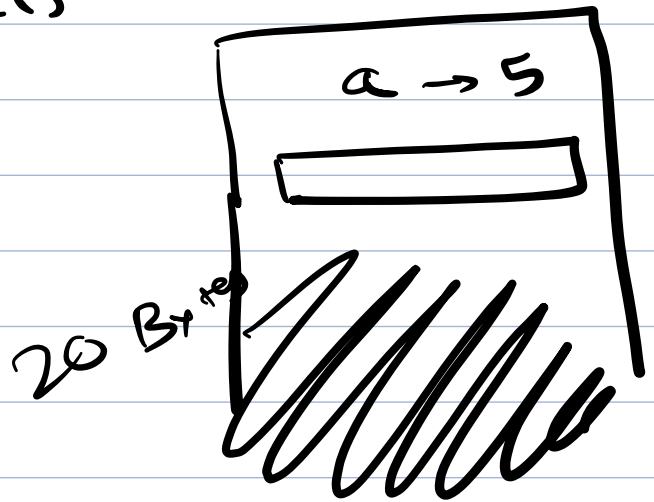
9 $\rightarrow 1$

10 $\rightarrow 1$

10 dc $\rightarrow 15$ ops

1 dc $\rightarrow \frac{3}{2}$ ops

1 dc $\rightarrow O(1)$



Java

```
ArrayList<String> a = new ArrayList<>()
a.add("50") // 50 is inserted at end
a.clear()
for (int i=0 ; i<a.size(); i++) <
    | System.out.print(a.get(i) + " ");
    >
```

Python

```
a = []
a.append("Orange") // Added orange at end
a.clear()
for i in range(len(a)):
    print(a[i])
```

C++

```
vector<int> a
a.push_back(60)
a.clear()
for (int i=0 ; i<a.size(); i++)
    cout << a[i]
```

- ① Declare a dynamic array
- ② Add an element
- ③ Access an element
- ④ Size or iterate